MASTER 2 DATA SCIENCE EN SANTÉ · SÉANCE 2 SUR 4

Orchestrer les pipelines de données de santé

Apache Airflow 3 : planifier, fiabiliser, surveiller



Antoine Teston
antoine.teston@univ-lille.fr
Université de Lille

Samedi 7 novembre 2026 · 8 h – 12 h

DÉROULÉ · 8 H – 12 H

Quatre parties, trois pauses

08:00 – 08:55	**01 · Orchestration**, concepts et premier DAG	55 min
08:55 – 09:05	Pause	10 min
09:05 – 10:00	**02 · Planifier** et faire communiquer les tâches	55 min
10:00 – 10:10	Pause	10 min
10:10 – 11:00	**03 · Fiabiliser**, surveiller, valider les données	50 min
11:00 – 11:10	Pause	10 min
11:10 – 12:00	**04 · Patterns avancés** et pipeline FHIR quotidien	50 min

OBJECTIFS

À l'issue de la séance, vous saurez

- Expliquer le rôle d'un orchestrateur
- Écrire un DAG Airflow 3 avec l'API TaskFlow
- Planifier : cron, intervalles, rattrapage
- Rendre un pipeline robuste : retries, idempotence, capteurs
- Surveiller et alerter : interface, Deadline Alerts, métriques
- Bloquer les données invalides avant l'entrepôt

AVANT DE COMMENCER · 8 H 00

Lancer l'installation maintenant

```
git clone https://github.com/Lille-Health-Data-Science/td-airflow-sante.git
```

A · Docker

Docker Desktop déjà installé

```
docker compose up airflow-init
docker compose up -d
```

≈ 1 Go à télécharger

B · Sans Docker

macOS ou Linux

```
scripts/installer_sans_docker.sh
scripts/lancer_sans_docker.sh
```

≈ 150 Mo à télécharger

C · Codespaces

Windows sans Docker

```
Code › Codespaces
› Create codespace
```

rien en local

C'est prêt quand le DAG **verification_installation** est vert. À midi : votre flux FHIR lancé de bout en bout.

LE MODULE

Le module en quatre séances

1 · Échanger
FHIR
17 oct.

des ressources FHIR

2 · Orchestrer
Airflow 3
7 nov.

un pipeline planifié

3 · Rechercher
OpenSearch
21 nov.

un index interrogeable

4 · Intégrer
atelier
12 déc.

une chaîne complète, évaluée

Le 16 octobre : entrepôt, ETL et OMOP. Ici, on branche l'entrepôt sur ses sources (FHIR), on automatise (Airflow) et on rend les textes interrogeables (moteur de recherche).

RAPPEL · SÉANCE 1

Rappel de la séance 1

FIN DU TP DE LA SÉANCE 1 · UN SCRIPT LANCÉ À LA MAIN

Interroger
l'API FHIR

Paginer
page par page

Aplatir
les glycémies

ET DEMAIN MATIN ?

« Et demain matin ? Qui le relance, que se passe-t-il si l'API ne répond pas, comment le savoir ? »

AUJOURD'HUI

Relancer, reprendre, prévenir : c'est le travail d'un orchestrateur, et l'objet de la séance.

01

Orchestration et concepts

Du script lancé à la main au graphe de tâches

08:00 – 08:55



Le défi des pipelines de données en santé

UNE CHAÎNE QUOTIDIENNE

1. Extraction du DPI
2. Transformation et nettoyage
3. Chargement dans l'entrepôt
4. Indicateurs qualité
5. Rapports

SANS ORCHESTRATEUR

- ⚠ Scripts lancés par cron
- ⚠ Pas de dépendances entre étapes
- ⚠ Échec invisible
- ⚠ Pas de reprise

Si l'extraction échoue, les étapes suivantes tournent quand même, sur des données périmées.

Qu'est-ce que l'orchestration ?

DÉFINITION

« Coordination automatisée de l'exécution de tâches selon un ordre défini »

Planification

quand lancer

Dépendances

dans quel ordre

Supervision

où en est-on

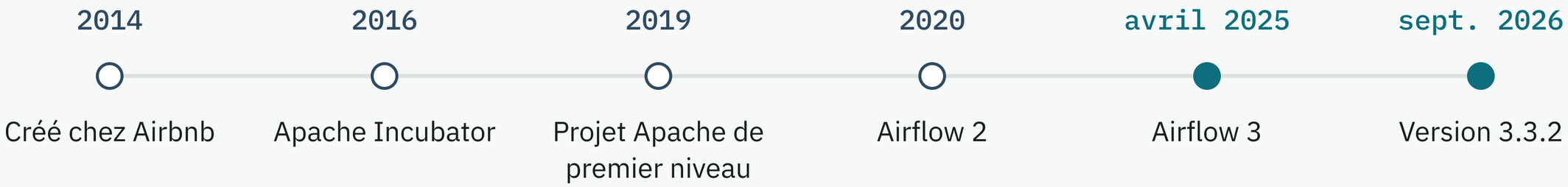
Gestion des erreurs

que faire si ça échoue

Historique / traçabilité

qui a tourné, quand

Pourquoi Apache Airflow ?



Workflows en Python

versionnés comme du code

Interface web

suivre, relancer, lire les journaux

Nombreux providers

bases, cloud, messageries

Passage à l'échelle

Celery, Kubernetes

Ce qui change avec Airflow 3

Task SDK

`from airflow.sdk import dag, task`

Assets

ex-Datasets : planifier par les données

Versionnage des DAG

les versions successives d'un DAG restent visibles

Backfills

gérés par le scheduler, depuis l'UI ou la CLI

Nouvelle interface

en React, sur l'API REST v2

Isolation

plus d'accès direct à la base de métadonnées depuis les tâches

⚠ Airflow 2 est en fin de vie depuis le 22 avril 2026 : beaucoup de tutoriels en ligne sont obsolètes.

Airflow et les alternatives

Critère	Cron	Airflow 3	Dagster	Prefect 3	Kestra
Dépendances	non	oui	oui	oui	oui
Modèle	horaire	tâches et assets	assets	fonctions Python	flux YAML
Interface web	non	oui	oui	oui	oui
Retry et alertes	non	oui	oui	oui	oui
Écosystème	—	très large	large	large	en croissance

Cron se contente de l'horaire ; un orchestrateur ajoute l'ordre, la reprise et l'historique.

Cas d'usage en santé

ETL hospitalier

DPI → transformation → entrepôt

Apprentissage automatique

collecte → prétraitement → entraînement → déploiement

Reporting

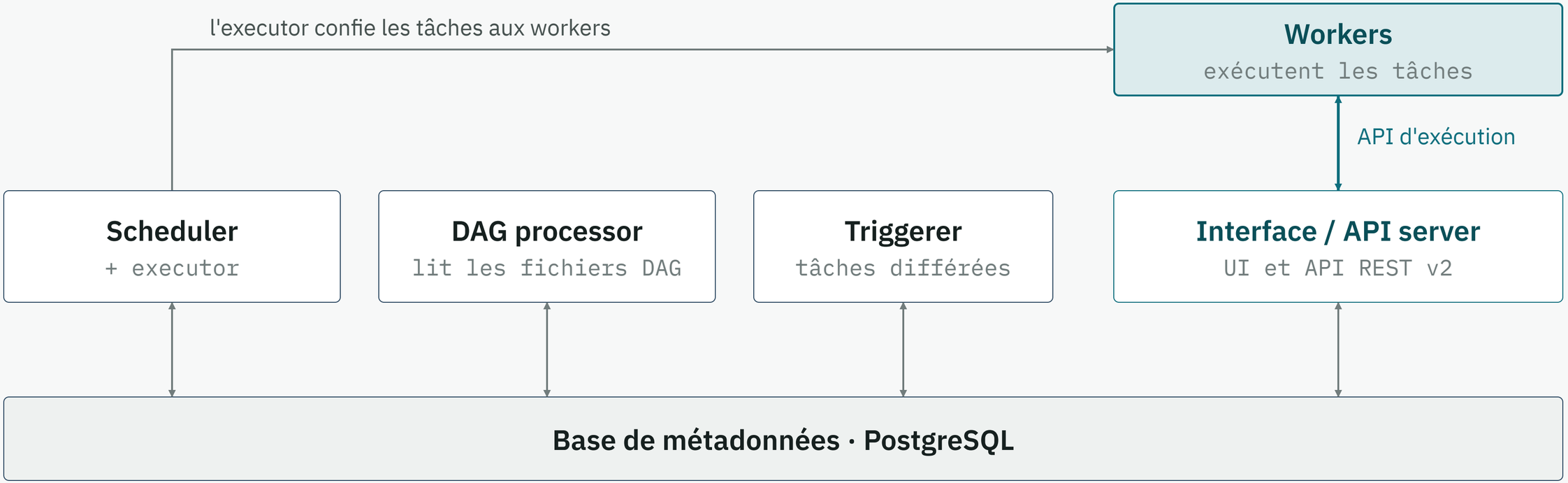
indicateurs → rapports → envoi

Surveillance épidémiologique

collecte → détection d'anomalies → alertes

Même structure partout : des étapes, un ordre, un horaire. Seul le contenu des tâches change.

Architecture d'Airflow 3



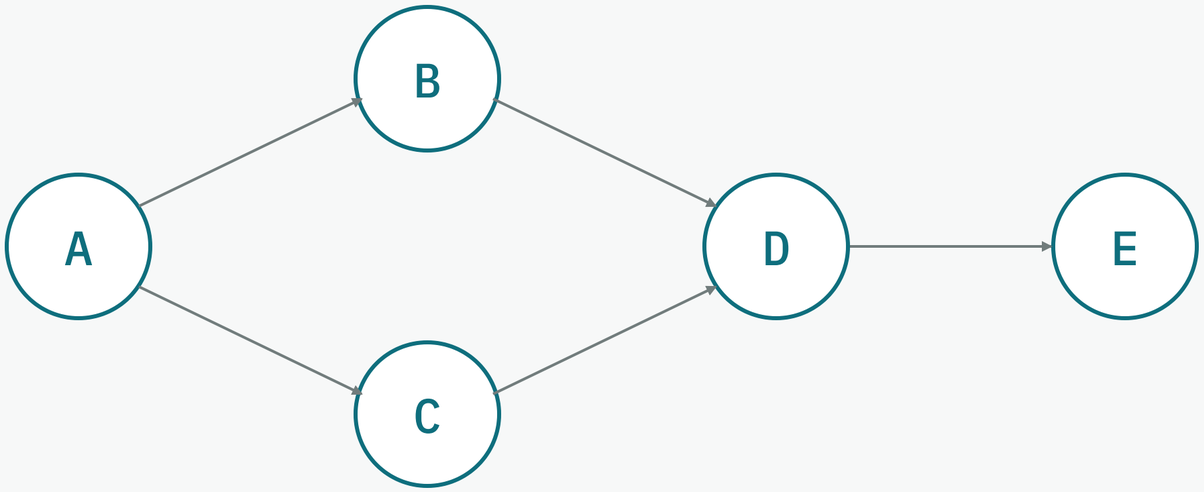
Les workers passent par l'API server : plus aucun accès direct à la base de métadonnées.

Les composants

Composant	Rôle
API server	Interface et API REST v2
Scheduler	Planifie, gère les dépendances
DAG processor	Analyse les fichiers Python des DAG
Executor	Local, Celery, Kubernetes, Edge
Triggerer	Attentes asynchrones
Base de métadonnées	États, historique

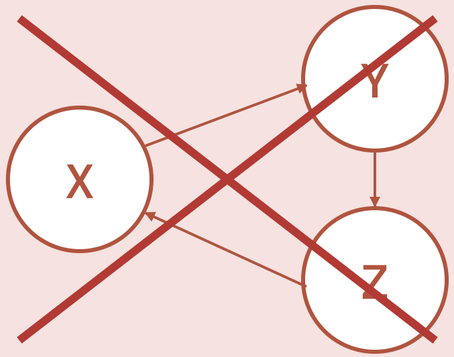
Docker Compose officiel : `postgres`, `redis`, `airflow-api-server`, `airflow-scheduler`, `airflow-dag-processor`, `airflow-worker`, `airflow-triggerer`, `airflow-init`

DAG : graphe orienté acyclique



Orienté : sens d'exécution défini

Acyclique : pas de boucle, donc pas de blocage



Cycle interdit : X attend Z, qui attend Y, qui attend X

Tâche et opérateur

Tâche : un nœud du DAG, une unité de travail exécutée et suivie

Opérateur : le modèle d'une tâche, ce qu'elle sait faire

Opérateur	Rôle	Module
`@task`	Fonction Python (API TaskFlow)	`airflow.sdk`
`PythonOperator`	Appeler une fonction Python	`airflow.providers.standard.operators.python`
`BashOperator`	Lancer une commande shell	`airflow.providers.standard.operators.bash`
`SQLExecuteQueryOperator`	Requête SQL, tout SGBD	`airflow.providers.common.sql.operators.sql`
`EmptyOperator`	Jonction, sans action	`airflow.providers.standard.operators.empty`
`Opérateurs des providers`	HTTP, Slack, SMTP…	`apache-airflow-providers-…`

Un premier DAG (API TaskFlow)

```python
from datetime import datetime
from airflow.sdk import dag, task

@dag(schedule="@daily", start_date=datetime(2026, 1, 1),
     catchup=False, tags=["initiation"])
def mon_premier_dag():
    @task
    def extraire() -> dict:
        return {"nb_patients": 150}

    @task
    def transformer(stats: dict) -> int:
        return stats["nb_patients"] * 2

    @task
    def charger(n: int):
        print(f"Chargement de {n} lignes")

    charger(transformer(extraire()))

mon_premier_dag()
```

Une fonction décorée = une tâche

Appels imbriqués = dépendances

Valeurs de retour transmises par XCom

La même chose avec des opérateurs

```
from airflow.sdk import DAG
from airflow.providers.standard.operators.python import PythonOperator
from airflow.providers.standard.operators.bash import BashOperator

with DAG(dag_id="etl_classique", schedule="@daily",
         start_date=datetime(2026, 1, 1), catchup=False):
    t1 = PythonOperator(task_id="extraire", python_callable=extraire)
    t2 = PythonOperator(task_id="transformer", python_callable=transformer)
    t3 = BashOperator(task_id="charger", bash_command="echo chargement")
    t1 >> t2 >> t3
```

Opérateurs importés du provider standard

task_id explicite pour chaque tâche

Ordre déclaré avec **>>**

Les fonctions extraire, transformer, charger sont définies ailleurs, sans décorateur

Déclarer les dépendances

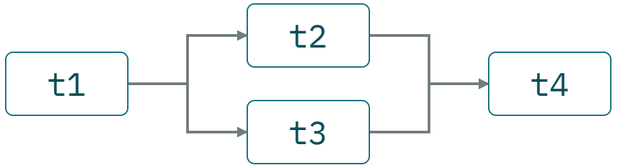
Séquentiel

```
t1 >> t2 >> t3
```



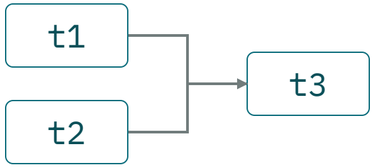
Parallèle

```
t1 >> [t2, t3] >> t4
```



Convergence

```
[t1, t2] >> t3
```



Fonction chain

```
from airflow.sdk import chain
chain(t1, t2, t3)
```



Installer Airflow 3 avec Docker Compose

```
git clone https://github.com/Lille-Health-Data-Science/td-airflow-sante.git
cd td-airflow-sante
docker compose up airflow-init    # 1re fois : image et base
docker compose up -d
```

Interface

http://localhost:8080
identifiants airflow / airflow

Mémoire

4 Go minimum pour Docker, 8 Go conseillés

Usage

Pour apprendre, pas pour la production

L'interface

Vues

- Liste des DAG
- Grid : historique des exécutions
- Graph : le graphe de tâches
- Gantt : durées
- Journaux de chaque tâche
- Versions du DAG

Actions

- Activer / mettre en pause
- Déclencher une exécution
- Lancer un backfill
- Effacer une tâche pour la relancer
- Marquer succès / échec

EXERCICE · 8 MIN

Votre premier DAG

1

Copier **exercices/mon_premier_dag.py** dans **dags/**

2

Le déclencher depuis l'interface

3

Lire le journal de la tâche **charger**

4

Ajouter une tâche **controler** entre **transformer** et **charger**, qui échoue si n < 100 (raise ValueError)

QUESTION

Que voit-on dans la vue Grid ?



Pause

08:55 – 09:05

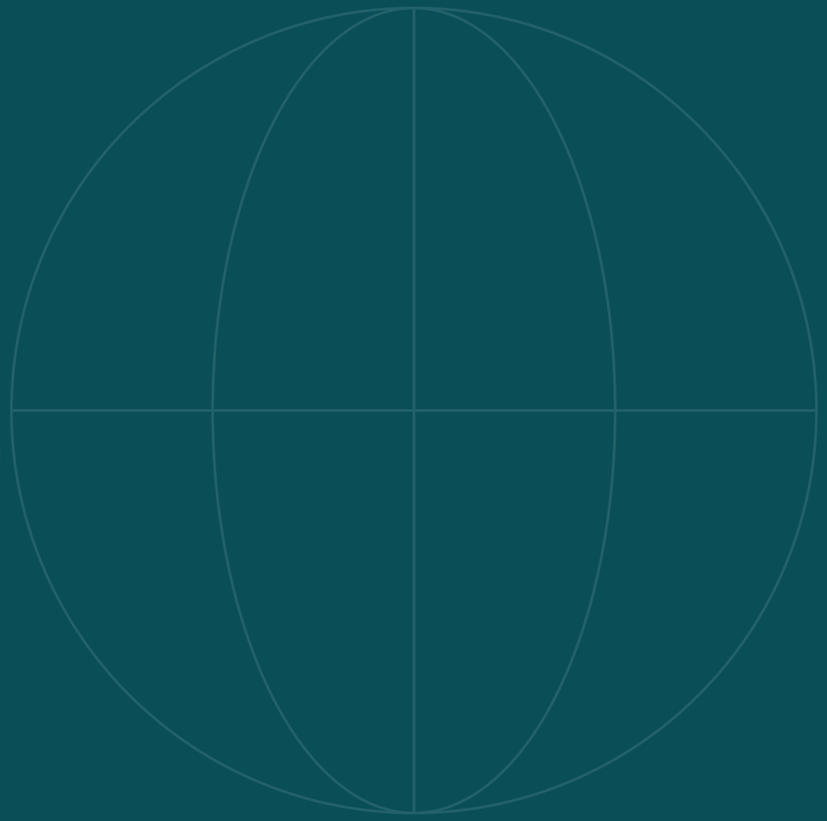
Partie 02 : quand et avec quelles données chaque tâche s'exécute.

02

Planifier et communiquer

Horaires, intervalles, rattrapage, XCom, assets

09:05 – 10:00



Les paramètres de planification

```
@dag(
    dag_id="etl_quotidien",
    schedule="0 6 * * *",            # tous les jours à 6 h
    start_date=datetime(2026, 1, 1),
    catchup=False,                    # défaut en Airflow 3
    max_active_runs=1,
    default_args={"owner": "data_team", "retries": 3,
                  "retry_delay": timedelta(minutes=5)},
)
```

schedule : cron, raccourci, timetable, assets ou None

max_active_runs=1 : une seule exécution à la fois

default_args : appliqués à toutes les tâches

> CORRECTION · AIRFLOW 3
>
> schedule_interval n'existe plus : un seul paramètre, schedule.

Expressions cron

CINQ CHAMPS, DE GAUCHE À DROITE

0	6	*	*	*
minute	heure	jour du mois	mois	jour de semaine

Expression	Signification
`0 6 * * *`	tous les jours à 6 h
`0 */2 * * *`	toutes les 2 heures
`0 8 * * 1-5`	du lundi au vendredi à 8 h
`0 0 1 * *`	le 1er de chaque mois à minuit

Raccourcis

Raccourci	Équivalent cron	Quand
`@once`	—	une seule exécution
`@hourly`	`0 * * * *`	toutes les heures
`@daily`	`0 0 * * *`	chaque jour à minuit
`@weekly`	`0 0 * * 0`	chaque dimanche à minuit
`@monthly`	`0 0 1 * *`	le 1er du mois à minuit
`@continuous`	—	relance dès la fin de l'exécution précédente
`None`	—	déclenchement manuel ou externe
`[asset, …]`	—	dès que les données sont prêtes

Date logique et intervalle de données : ce qui change

DAG quotidien, exécution déclenchée le **15 janvier 2026 à 00:00**

AIRFLOW 2

CronDataIntervalTimetable

logical_date	14/01 00:00
intervalle	14/01 → 15/01

« traite les données du 14 »

AIRFLOW 3 · PAR DÉFAUT

CronTriggerTimetable

logical_date	15/01 00:00
intervalle	vide

début = fin = 15/01 00:00

CORRECTION · AIRFLOW 3

execution_date n'existe plus : utiliser logical_date et data_interval_start / data_interval_end.

Retrouver un intervalle explicite

```
from datetime import timedelta
from airflow.timetables.trigger import CronTriggerTimetable

@dag(schedule=CronTriggerTimetable("0 6 * * *",
                                   timezone="Europe/Paris",
                                   interval=timedelta(days=1)), ...)
def etl_quotidien():
    @task
    def extraire(data_interval_start=None, data_interval_end=None):
        print(f"Données du {data_interval_start} au {data_interval_end}")
```

L'exécution de 6 h traite les 24 h précédentes

Fuseau horaire explicite

Rejouer un jour = retraiter exactement le même intervalle

Rattrapage et backfill

RATTRAPAGE AUTOMATIQUE

catchup=True

À l'activation, crée toutes les exécutions manquées depuis start_date

catchup=False

Seulement la plus récente. Défaut en Airflow 3, recommandé

BACKFILL À LA DEMANDE

```
airflow backfill create --dag-id etl_quotidien \
    --from-date 2026-01-01 --to-date 2026-01-31 \
    --reprocess-behavior failed --max-active-runs 2
```

ou depuis l'interface : **Trigger › Backfill**

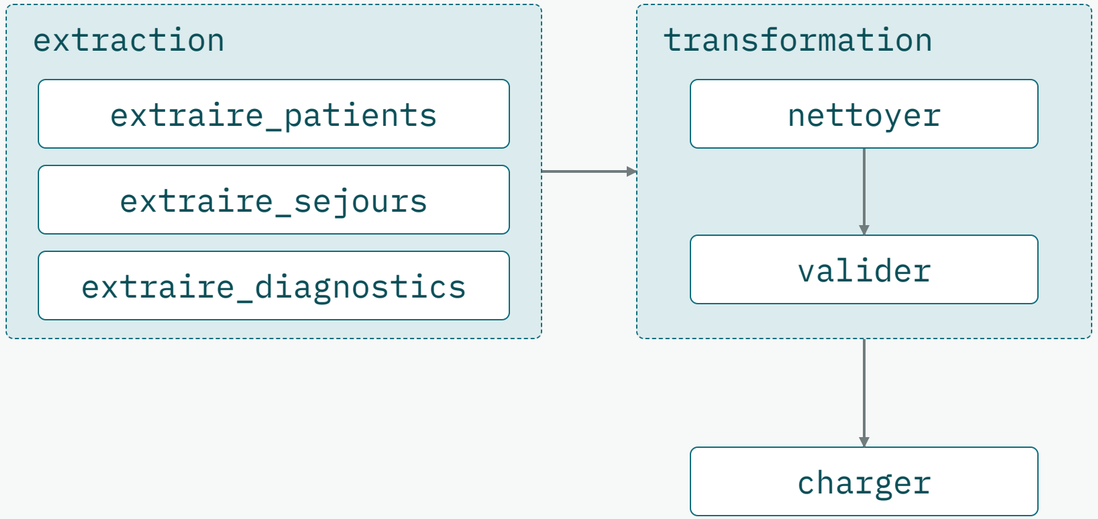
Regrouper les tâches

```python
from airflow.sdk import task_group

@task_group
def extraction():
    extraire_patients()
    extraire_sejours()
    extraire_diagnostics()

@task_group
def transformation():
    nettoyer() >> valider()

extraction() >> transformation() >> charger()
```



Un groupe se replie dans la vue Graph

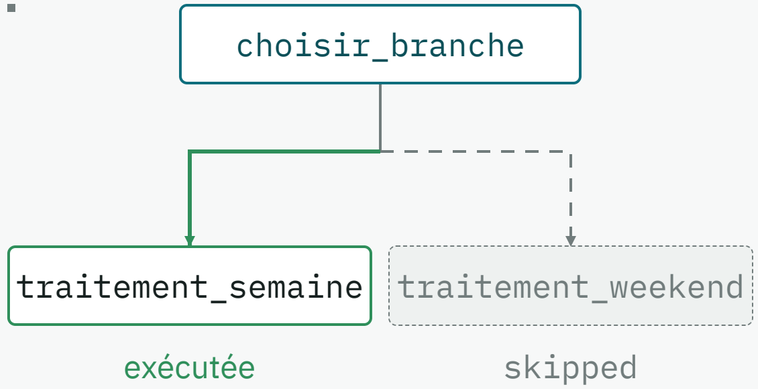
Branches conditionnelles

```python
@task.branch
def choisir_branche(logical_date=None) -> str:
    if logical_date.weekday() < 5:
        return "traitement_semaine"
    return "traitement_weekend"

@task
def traitement_semaine(): ...
@task
def traitement_weekend(): ...

choisir_branche() >> [traitement_semaine(), traitement_weekend()]
```

EXEMPLE : UN MARDI



La fonction renvoie le task_id de la branche à suivre

XCom : passer des informations entre tâches

API TASKFLOW

La valeur de retour d'une tâche **est** un XCom, transmis automatiquement

```
charger(transformer(extraire()))
```

STYLE CLASSIQUE

On lit l'XCom à la main, via l'instance de tâche **ti**

```
def transformer(ti):
    stats = ti.xcom_pull(task_ids="extraire", key="return_value")
```

CORRECTION · AIRFLOW 3

En Airflow 3, xcom_pull() sans task_ids ne lit que la tâche courante : toujours préciser la tâche source.

XCom : ce qu'on y met, ce qu'on n'y met pas

Un XCom est **sérialisé** et **stocké en base** : il doit rester petit.

BON

```
return {"chemin": "/data/sortie/resultat.parquet",
        "lignes": 1500}
```

Un chemin et un compte : quelques octets

MAUVAIS

```
return df.to_dict()
```

Tout le DataFrame écrit dans la base

POINT CLÉ

Gros volumes : écrire un fichier (Parquet), passer le chemin.

Gabarits Jinja

```
BashOperator(
    task_id="traiter",
    bash_command="python script.py --date {{ ds }} --debut {{ data_interval_start }}")
```

Gabarit	Valeur ou sens
{{ ds }}	2026-01-15
{{ ds_nodash }}	20260115
{{ logical_date }}	date logique de l'exécution
{{ data_interval_start }} / {{ data_interval_end }}	bornes de l'intervalle
{{ params.seuil }}	paramètre de l'exécution
{{ var.value.ma_variable }}	variable globale

Paramètres et variables

```python
from airflow.sdk import Param, Variable

@dag(params={"seuil": Param(100, type="integer", minimum=0),
             "environnement": "prod"}, ...)
def controle():
    @task
    def verifier(params=None):
        seuil_global = Variable.get("seuil_alerte", default=50)
        print(params["seuil"], seuil_global)
```

Params

Par exécution : saisis au déclenchement, validés

Variables

Globales : UI, CLI, variables d'environnement AIRFLOW_VAR_…

Jamais de mot de passe dans une variable : utiliser les **Connections**

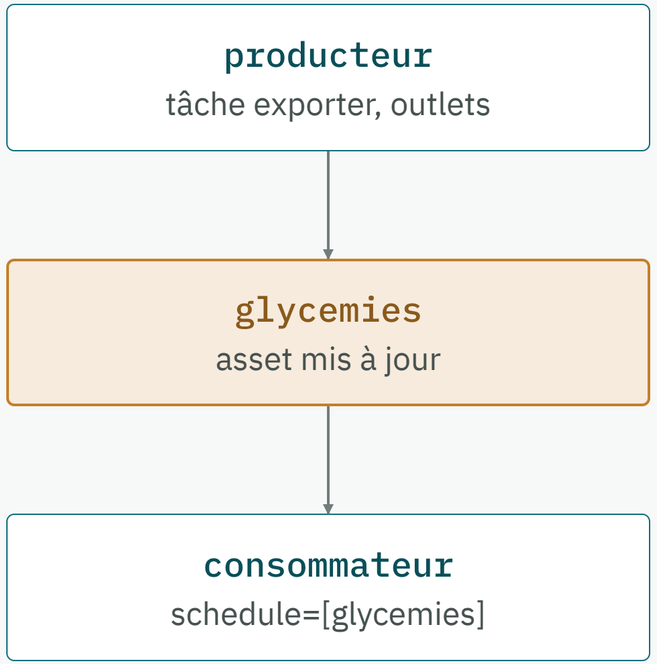
Assets : déclencher quand les données sont prêtes

```python
from airflow.sdk import Asset, dag, task

glycemies = Asset("s3://entrepot/fhir/glycemies.parquet")

@dag(schedule="@daily", start_date=datetime(2026, 11, 1), catchup=False)
def producteur():
    @task(outlets=[glycemies])
    def exporter(): ...

@dag(schedule=[glycemies])
def consommateur():
    @task
    def calculer_indicateurs(): ...
```



EXERCICE · 6 MIN

Prévoir les exécutions

LE DAG

`schedule="0 6 * * *"` `start_date=datetime(2026, 11, 1)` `catchup=True`

Activé le **4 novembre à 10 h**, Airflow 3 avec ses réglages par défaut.

1

Combien d'exécutions sont créées immédiatement ?

2

Quelle logical_date pour la première ?

3

Même question avec interval=timedelta(days=1)

4

Que change catchup=False ?

Pause

10:00 – 10:10

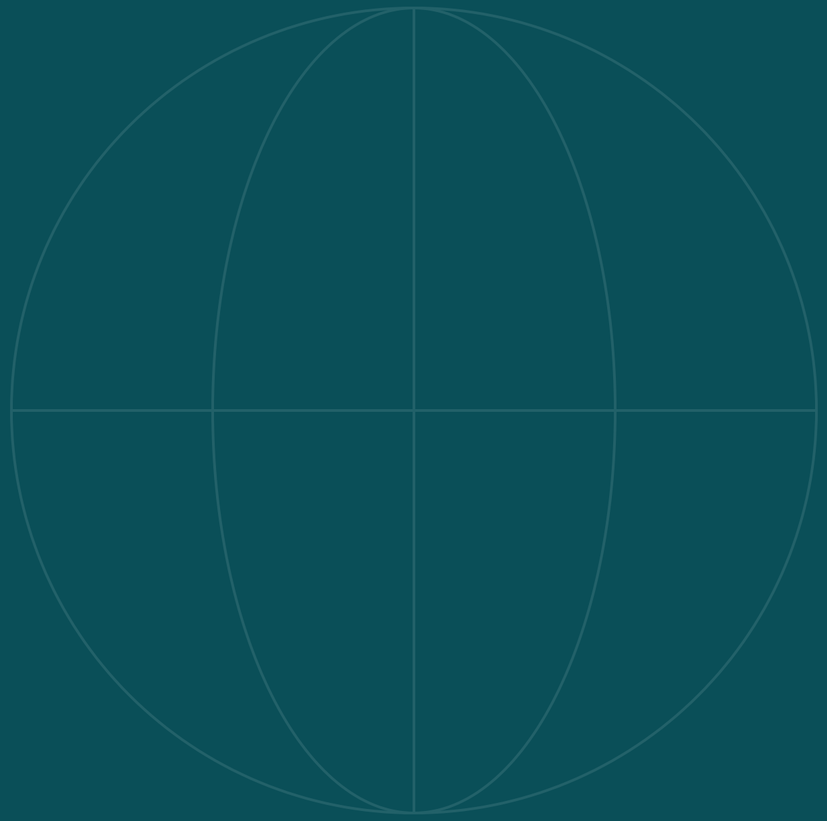
Partie 03 : que faire quand ça échoue, et comment le savoir.

03

Fiabiliser et surveiller

Retries, capteurs, idempotence, alertes, qualité

10:10 – 11:00



Relancer automatiquement

```
default_args = {
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "max_retry_delay": timedelta(minutes=30),
}

@task(retries=5, retry_delay=timedelta(seconds=30))
def appeler_api_fhir(): ...
```

default_args : pour toutes les tâches du DAG

@task(retries=…) : réglage propre à une tâche

Délai croissant, plafonné par max_retry_delay

> ⚠ Un retry ne sert qu'aux erreurs transitoires (réseau, API surchargée), pas aux erreurs de données.

Réagir aux événements : callbacks et notifiers

```
from airflow.providers.slack.notifications.slack_webhook import (
    SlackWebhookNotifier)

alerte = SlackWebhookNotifier(
    slack_webhook_conn_id="slack_alertes",
    text="Échec de {{ ti.task_id }} dans {{ dag.dag_id }}")

@task(on_failure_callback=alerte,
      on_retry_callback=lambda ctx: print("Nouvelle tentative"))
def tache_critique(): ...
```

Un notifier = une alerte prête à l'emploi

Le texte est un gabarit Jinja

Le webhook est rangé dans une Connection

Mêmes callbacks possibles au niveau du DAG

Capteurs : attendre une condition

```python
from airflow.providers.standard.sensors.filesystem import FileSensor

attendre = FileSensor(
    task_id="attendre_fichier",
    filepath="/data/entree/patients_{{ ds }}.csv",
    poke_interval=60,        # vérifier toutes les 60 s
    timeout=3600,            # abandon après 1 h
    mode="reschedule",       # libère le worker entre deux essais
)
attendre >> traiter_donnees()
```

AUTRES CAPTEURS

ExternalTaskSensor
attendre un autre DAG

HttpSensor
attendre une API

Les opérateurs différés (deferrable) attendent dans le triggerer sans occuper de worker.

Idempotence : rejouer sans doublonner

MAUVAIS `INSERT INTO sejours VALUES (...)` exécutions multiples = doublons

BON

```
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator

charger_jour = SQLExecuteQueryOperator(
    task_id="charger_jour", conn_id="entrepot",
    sql=["DELETE FROM sejours WHERE jour = %(jour)s",
         "INSERT INTO sejours SELECT * FROM stg_sejours WHERE jour = %(jour)s"],
    parameters={"jour": "{{ ds }}"},
)
```

Règles de déclenchement

```python
@task(trigger_rule="all_done")
def nettoyer_fichiers_temp(): ...
```

Nettoyer même si une tâche amont a échoué

Par défaut, une tâche attend le succès de toutes ses tâches amont.

Règle	La tâche démarre si les tâches amont…
`all_success` (défaut)	toutes ont réussi
`all_failed`	toutes ont échoué
`all_done`	toutes sont terminées, quel que soit l'état
`one_success`	au moins une a réussi
`one_failed`	au moins une a échoué
`none_failed`	aucune n'a échoué (succès ou sautée)
`none_failed_min_one_success`	aucun échec, au moins un succès
`all_done_min_one_success` (3.1)	toutes terminées, au moins un succès

Deadline Alerts : remplacer les SLA

```python
from datetime import timedelta
from airflow.sdk import DAG, DeadlineAlert, DeadlineReference, AsyncCallback
from airflow.providers.slack.notifications.slack_webhook import SlackWebhookNotifier

with DAG(
    dag_id="etl_critique",
    deadline=DeadlineAlert(
        reference=DeadlineReference.DAGRUN_QUEUED_AT,
        interval=timedelta(hours=2),
        callback=AsyncCallback(SlackWebhookNotifier,
            kwargs={"text": "{{ dag_run.dag_id }} : échéance dépassée"}),
    ),
):
    ...
```

Référence + délai + action

Ici : 2 h après la mise en file

Expérimental depuis 3.1

Page Deadlines dans l'interface (3.3)

Superviser dans l'interface

Liste des DAG

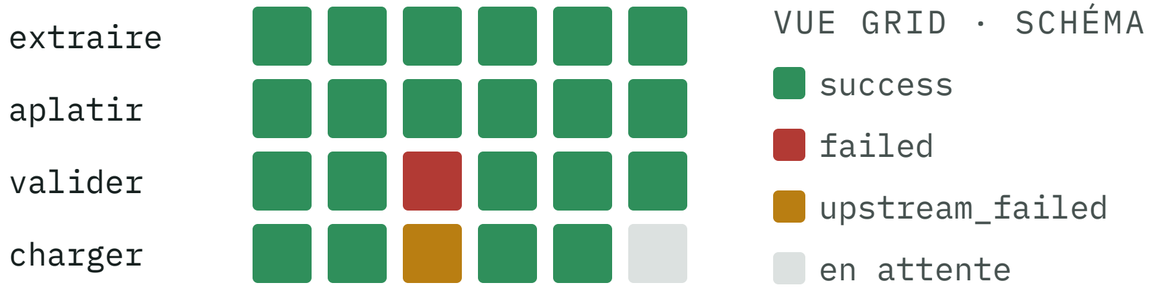
actif / en pause, dernière exécution, prochaine

Grid et Graph

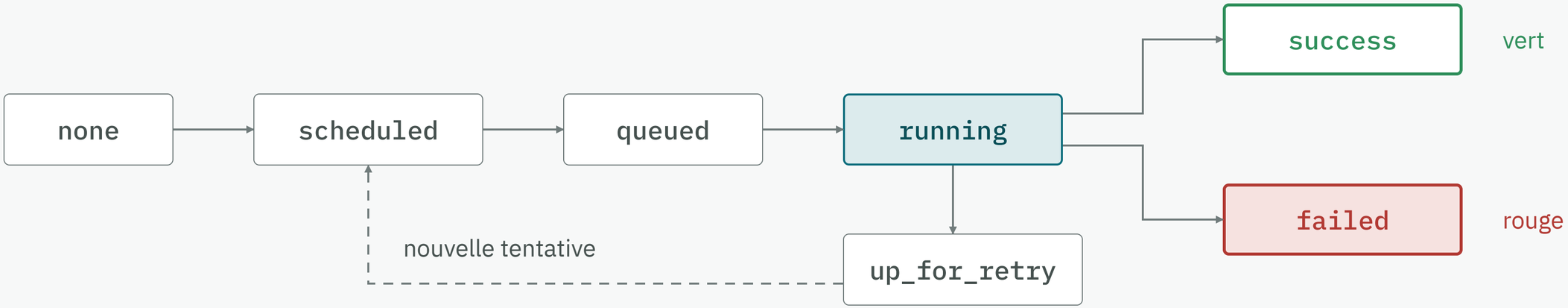
état de chaque tâche, journaux en un clic

Gantt

durées, goulots d'étranglement



Les états d'une tâche



up_for_reschedule
capteur en mode reschedule

deferred
attente dans le triggerer

skipped
branche non choisie

upstream_failed
une tâche amont a échoué

Alertes et métriques

ALERTES

E-mail

provider SMTP, email_on_failure

Messagerie

Slack, Teams, via les notifiers

MÉTRIQUES · OPENTELEMETRY

```
# docker-compose.yaml (extrait)
AIRFLOW__METRICS__OTEL_ON: "true"
AIRFLOW__METRICS__OTEL_HOST: otel-collector
AIRFLOW__METRICS__OTEL_PORT: "4318"
```

DAG actifs, durées des tâches, taux d'échec, file du scheduler → **Prometheus / Grafana**

Pourquoi valider les données ?

PROBLÈMES FRÉQUENTS

- ⚠ Fichier vide ou corrompu
- ⚠ Schéma modifié sans prévenir
- ⚠ Valeurs aberrantes ou manquantes
- ⚠ Doublons

CONSÉQUENCES

- Erreurs silencieuses
- Rapports faux
- Décisions prises sur des données fausses

💡 Une tâche verte dit que le code a tourné, pas que les données sont justes.

Valider un DataFrame avec Pandera

```python
import pandas as pd
import pandera.pandas as pa

schema = pa.DataFrameSchema({
    "patient_id": pa.Column(str, unique=True),
    "age": pa.Column(int, pa.Check.between(0, 120)),
    "date_admission": pa.Column("datetime64[ns]", nullable=False),
})

@task
def valider(chemin: str) -> dict:
    df = pd.read_parquet(chemin)
    if df.empty:
        raise ValueError("Fichier vide")
    schema.validate(df, lazy=True)   # liste toutes les erreurs
    return {"chemin": chemin, "lignes": len(df)}
```

Schéma déclaratif : types, bornes, unicité, valeurs manquantes

lazy=True : toutes les erreurs d'un coup

Une exception fait échouer la tâche, donc arrête la chaîne

Mêmes règles que l'exemple d'origine (âge 0–120, unicité, dates présentes)

Outils de validation

Pandera

Schémas de DataFrames pandas ou polars, léger

GX Core

Great Expectations 1.x : suites d'attentes, rapports

Tests dbt

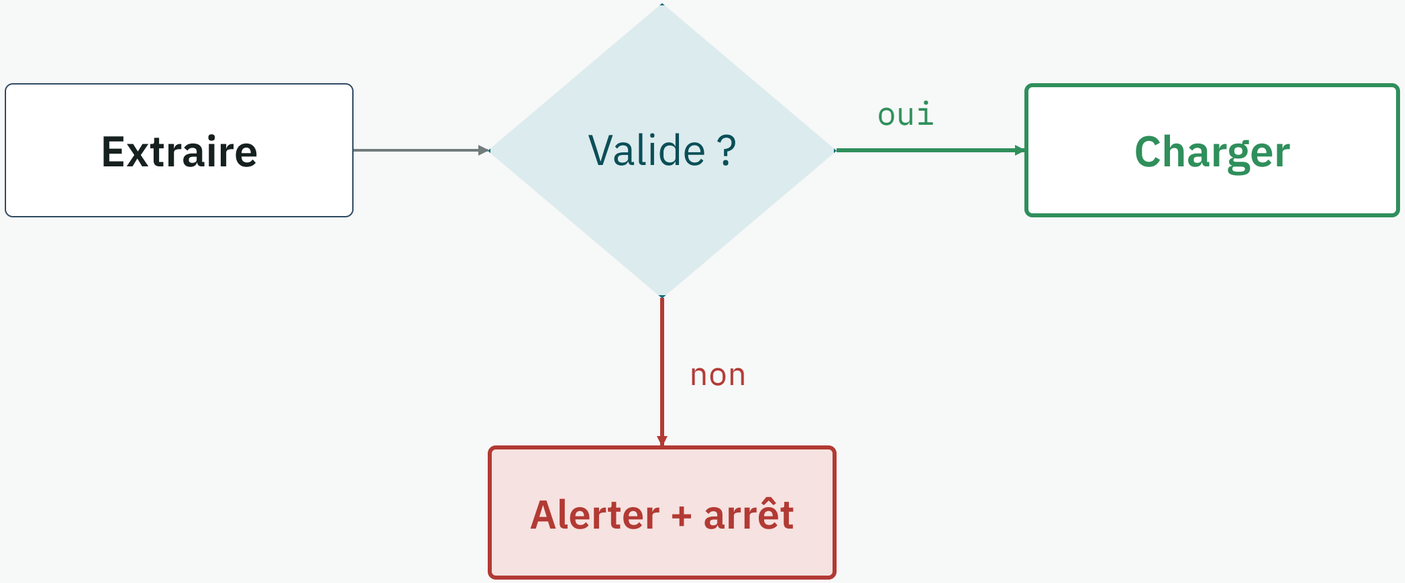
Dans la couche SQL, vus le 16 octobre

Soda Core

Contrôles déclaratifs

Choisir selon l'endroit où vit la donnée : un DataFrame dans une tâche Python, ou une table dans l'entrepôt.

La validation comme porte



Les données invalides ne passent jamais en production.

Dans Airflow : valider échoue, charger passe en upstream_failed, le callback d'échec alerte.



Pause

11:00 – 11:10

Partie 04 : des patterns réutilisables, appliqués au flux FHIR.

04

Patterns avancés et application

Mapping dynamique, pools, opérateurs maison, FHIR

11:10 – 12:00



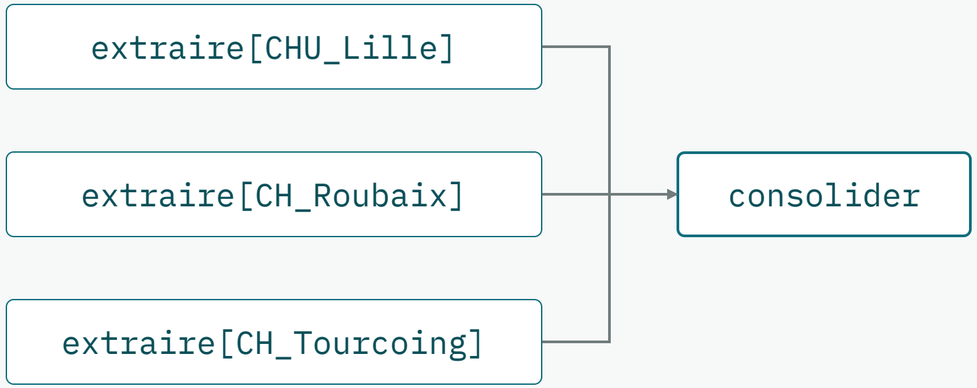
Tâches dynamiques : une tâche par établissement

```python
HOPITAUX = ["CHU_Lille", "CH_Roubaix", "CH_Tourcoing"]

@task
def extraire(hopital: str) -> str:
    return f"/data/brut/{hopital}.parquet"

@task
def consolider(chemins: list[str]):
    print(f"{len(chemins)} fichiers à fusionner")

consolider(extraire.expand(hopital=HOPITAUX))
```



expand() crée une instance de tâche par élément

La liste peut venir d'une tâche précédente

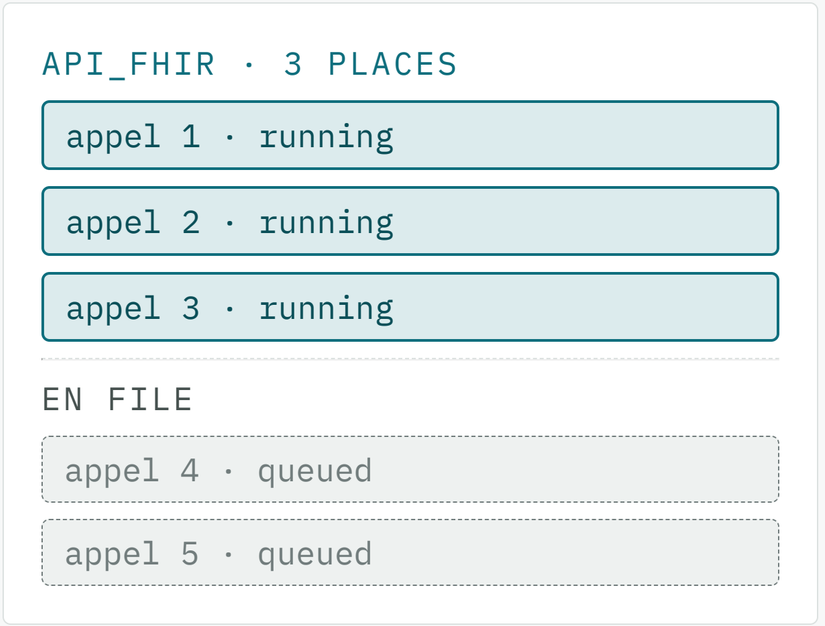
Pools : limiter la concurrence

CRÉER LE POOL

```
airflow pools set api_fhir 3 "Limite d'appels à l'API FHIR"
```

L'UTILISER

```
@task(pool="api_fhir", pool_slots=1)
def appeler_api(): ...
```



POINT CLÉ

Ne pas saturer une API ou une base de production.

Un opérateur maison réutilisable

```python
from airflow.sdk import BaseOperator
from airflow.providers.postgres.hooks.postgres import PostgresHook

class ControleQualiteOperator(BaseOperator):
    def __init__(self, table, controles, conn_id="entrepot", **kwargs):
        super().__init__(**kwargs)
        self.table, self.controles, self.conn_id = table, controles, conn_id

    def execute(self, context):
        hook = PostgresHook(postgres_conn_id=self.conn_id)
        for nom, sql in self.controles.items():
            n = hook.get_first(sql.format(table=self.table))[0]
            if n > 0:
                raise ValueError(f"{nom} : {n} lignes en erreur")
```

Hériter de **BaseOperator**

Toute la logique dans **execute(context)**

Réutilisable dans tous les DAG



Base de l'entrepôt : permise. Base de métadonnées d'Airflow : interdite.

Application : le flux FHIR quotidien

Extraire
API FHIR, _lastUpdated ≥ début d'intervalle

`pool + retries`

Aplatir
vue tabulaire, Parquet

`XCom de chemin`

Valider
Pandera

`porte`

Charger
DuckDB ou PostgreSQL

`idempotence`

Publier
l'asset glycemies

`outlets`

Chaque brique a été vue ce matin : le flux FHIR les assemble en un seul DAG quotidien.

Le DAG correspondant

```python
@dag(schedule=CronTriggerTimetable("0 6 * * *", timezone="Europe/Paris",
                                   interval=timedelta(days=1)),
     start_date=datetime(2026, 11, 1), catchup=False, tags=["fhir"])
def fhir_glycemies_quotidien():
    @task(pool="api_fhir", retries=5, retry_delay=timedelta(minutes=2))
    def extraire(data_interval_start=None) -> str: ...   # NDJSON
    @task
    def aplatir(chemin: str) -> str: ...                 # Parquet
    @task
    def valider(chemin: str) -> str: ...                 # Pandera
    @task(outlets=[glycemies])
    def charger(chemin: str): ...                        # DELETE + INSERT du jour

    charger(valider(aplatir(extraire())))

fhir_glycemies_quotidien()
```

Intervalle explicite d'un jour

Pool et retries sur l'appel réseau

Asset publié en sortie

TP · 20 MIN

Construire le flux FHIR

1. Compléter **extraire** avec le code de pagination de la séance 1 (paramètre _lastUpdated=ge{data_interval_start:%Y-%m-%d})
2. Écrire **aplatir** vers Parquet
3. Ajouter la validation : unités autorisées g/L, mg/dL, mmol/L
4. Charger dans DuckDB de façon idempotente
5. Provoquer une panne (mauvaise URL) : 3 retries à 20 s, puis l'alerte

À VÉRIFIER

Grid : quatre tâches vertes

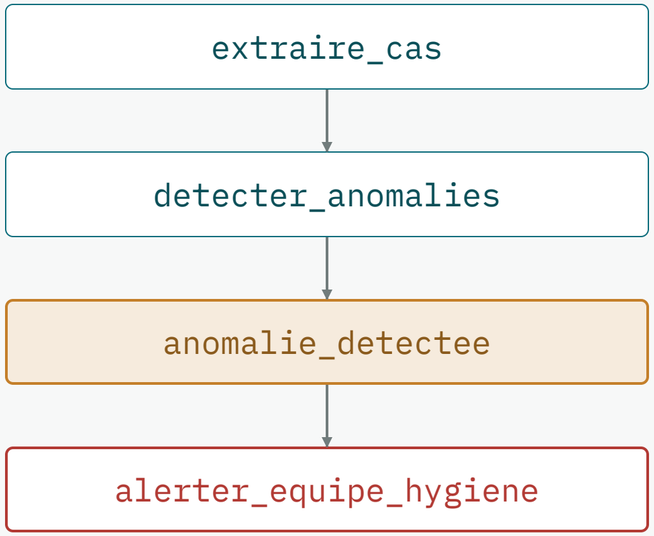
Relancer le même jour : aucun doublon

Panne : up_for_retry, failed, ligne dans alertes.log

Sans réseau : Trigger avec source = fichier

Autre exemple : surveillance épidémiologique

```
@dag(schedule="0 8 * * *", start_date=datetime(2026, 11, 1),
     catchup=False, tags=["epidemio", "alertes"])
def surveillance_infections():
    @task
    def extraire_cas(ds=None):
        return compter_cas_journaliers(ds)
    @task
    def detecter_anomalies(cas):
        return cusum(cas, seuil=5)        # ex. méthode CUSUM
    @task.short_circuit
    def anomalie_detectee(anomalies):
        return len(anomalies) > 0
    alerte = alerter_equipe_hygiene()
    anomalie_detectee(detecter_anomalies(extraire_cas())) >> alerte
```



Rien d'anormal : l'alerte est sautée

compter_cas_journaliers, cusum, alerter_equipe_hygiene : pseudo-code

Bonnes pratiques en santé

Traçabilité

Journaux d'exécution

Historique des exécutions

Versions de DAG

Documentation des transformations

Sécurité

Connections et secrets backend

Jamais d'identifiant en clair

Droits gérés par rôles

Hébergement HDS si données de santé

Pas de donnée patient dans les journaux

Robustesse

Tester les DAG avant production

Contrôles qualité

Idempotence

Prévoir les cas d'erreur

POINT CLÉ

Un print(df) dans une tâche écrit des données de santé dans les journaux.

Architecture de référence

Composant	Technologie	Rôle
Orchestration	Airflow 3	Planification, supervision
Stockage brut	Stockage objet compatible S3	Lac de données
Formats	Parquet, Iceberg	Tables versionnées
Traitement	Python (Polars), DuckDB, Spark	Transformations
Entrepôt	PostgreSQL, DuckDB	Données agrégées, OMOP
Qualité	Pandera, GX Core, tests dbt	Validation
Visualisation	Superset, Metabase	Tableaux de bord

ÉVALUATION · WOOCLAP · 4 MIN

QCM : quatre questions sur Wooclap

REJOINDRE

wooclap.com

Code de l'événement

YNZKYOP

1. Ouvrez wooclap.com sur votre téléphone, ou scannez le QR code
2. Saisissez le code de l'événement

`schedule` `Deadline Alerts` `XCom` `idempotence`

SYNTHÈSE

À retenir

- Un DAG décrit quoi lancer, dans quel ordre et que faire en cas d'échec
- Airflow 3 : airflow.sdk, assets, backfills et versions gérés par le scheduler
- Une exécution traite un intervalle explicite : on peut la rejouer
- Idempotence, retries et validation rendent la chaîne fiable
- On surveille et on alerte, sans exposer de données de santé

POUR LA DISCUSSION

Dans votre stage, quel script lancé à la main mériterait un DAG, et quelle serait sa première alerte ?

PROCHAINE SÉANCE · SAMEDI 21 NOVEMBRE

Rechercher dans les comptes rendus : moteur plein texte et recherche sémantique

SÉANCE 3 · 8 H – 12 H

Rechercher dans les comptes rendus

OpenSearch 3.8 (Lucene 10), EDS-NLP

⚠ À préparer : Docker, 4 Go de mémoire libres pour OpenSearch.

POUR ALLER PLUS LOIN

Ressources

Documentation Airflow
`airflow.apache.org/docs`

Guide de migration vers Airflow 3
`airflow.apache.org › upgrading_to_airflow3`

Guides Astronomer
`astronomer.io/docs/learn`

Pandera
`pandera.readthedocs.io`

GX Core
`docs.greatexpectations.io`

Dagster, Prefect, Kestra
`dagster.io · prefect.io · kestra.io`

Contact : **Antoine Teston**, antoine.teston@univ-lille.fr

Notions, exemples et exercices adaptés des supports d'Antoine Lamer (Université de Lille – ILIS, ULR 2694 METRICS), mis à jour en 2026.